

Università degli Studi di Padova – Dipartimento di Ingegneria dell'Informazione
Corso di Laurea in Ingegneria Informatica

Relazione per la prova finale
***«Progettazione e sviluppo di un software in C++ per l'analisi
interattiva del problema della brachistocrona»***

Relatore: Prof. Stefano Ghidoni
Correlatore: Prof.ssa Laura Caravenna

Laureando: Mattia Zanini
Matricola n.: 2101840

Padova, 21/07/2026

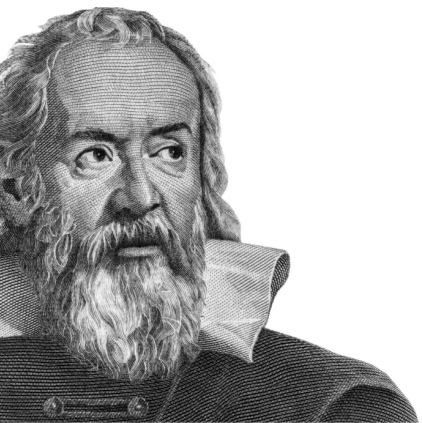
IL PROBLEMA DELLA BRACHISTOCRONA

« *...lationem omnium velocissimam [...] non per brevissimam
lineam [...] sed per circuli portionem, fieri* »

Galileo Galilei, Discorsi e dimostrazioni matematiche intorno a due nuove scienze attenenti alla meccanica & i movimenti locali, Leida, Elsevier, 1638, Giornata terza, Scholium dopo Theorema XXII, Propositio XXXVI

Il Quesito: Dati due punti **A** e **B** a quote differenti, qual è la traiettoria che un corpo deve seguire per andare da **A** a **B** nel minor tempo possibile sotto l'effetto della sola gravità?

La Soluzione: L'arco di **cicloide**. Nel caso della mezza arcata di cicloide, il tempo di percorrenza risulta circa il **16%** inferiore rispetto a quello lungo la linea retta tra gli **stessi estremi**.



LA SOLUZIONE ANALITICA

Minimizzazione del funzionale del tempo totale di discesa:

$$T(u) = \frac{1}{\sqrt{2g}} \int_0^L \sqrt{\frac{1 + |u'(x)|^2}{u(x)}} dx$$

Risolvendo tramite le equazioni di Eulero-Lagrange, si giunge alla soluzione parametrica dell'arco di **cicloide**:

$$\begin{cases} x(t) = c(t - \sin t) \\ y(t) = c(1 - \cos t) \end{cases} \quad t \in [0, \tau]$$

Soluzione parametrica dell'arco di cicloide

$$\begin{cases} L = x_B = c(\tau - \sin \tau) \\ y_B = c(1 - \cos \tau) \end{cases}$$

Condizioni al contorno nel punto di arrivo **B**

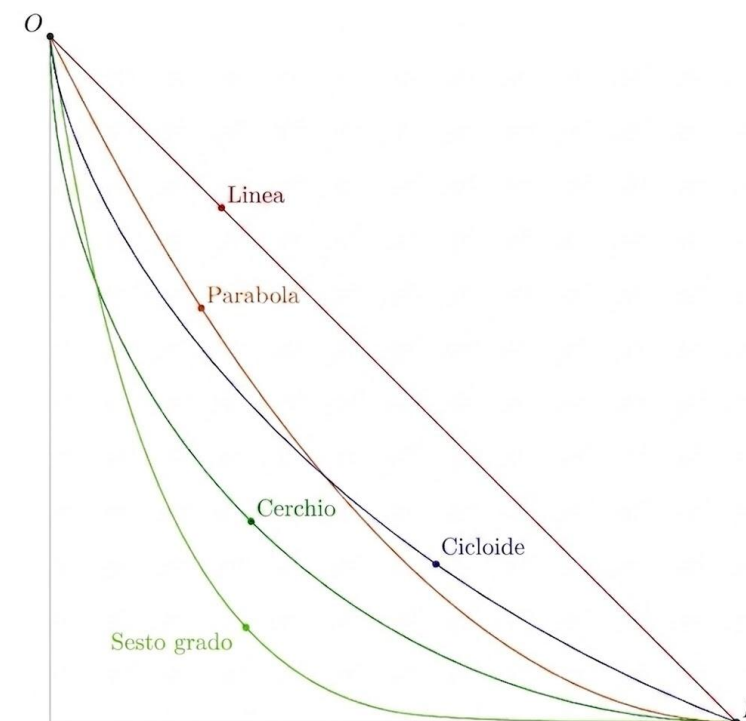


Illustrazione comparativa di traiettorie di caduta rappresentabili come grafici di una funzione $y = -u(x)$

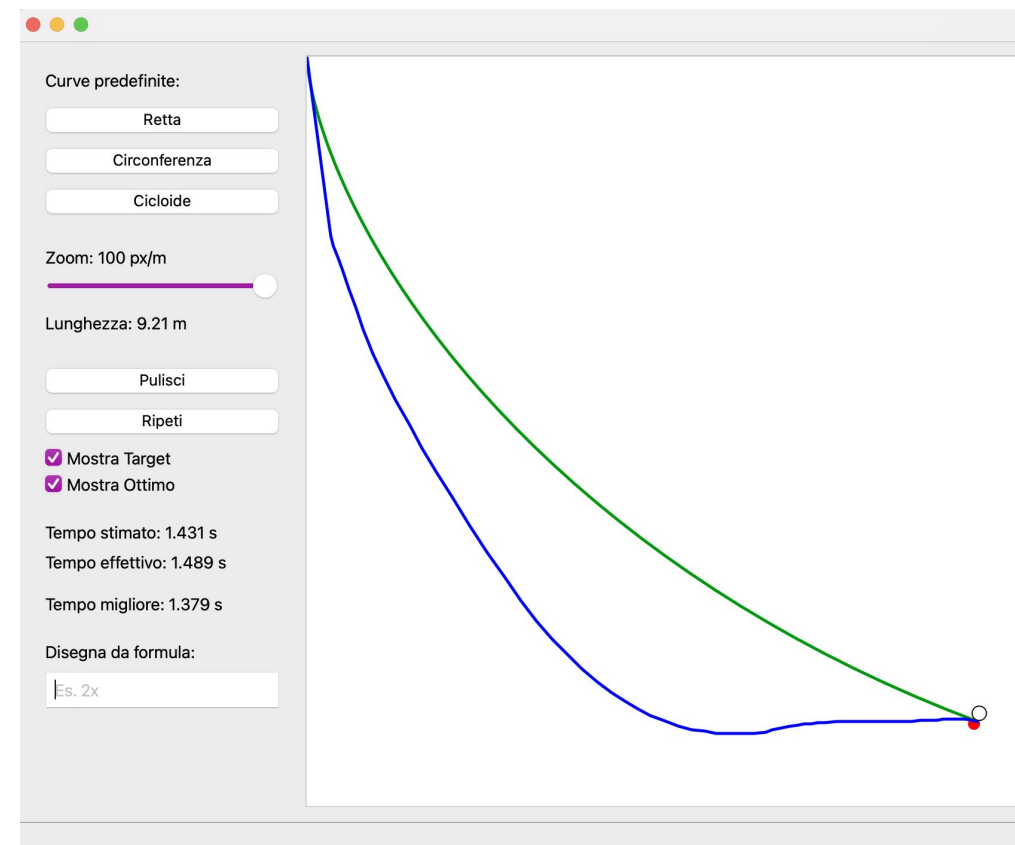
Il progetto crea un'esperienza interattiva digitale partendo dal problema della Brachistocrona

Sperimentazione Interattiva

Consente all'utente di tracciare curve arbitrarie sia tramite disegno libero a mano sia digitando equazioni matematiche

Visualizzazione Quantitativa

Visualizza numericamente e graficamente lo scostamento temporale in tempo reale fra il tragitto provato dall'utente e la cicloide ideale calcolata dal sistema.



DESCRIZIONE DEL LAVORO

DIPENDENZE E LIBRERIE

Libreria	Ruolo Principale nel Progetto		Caratteristica Chiave Sfruttata
 Qt 6.10+	Infrastruttura GUI, gestione eventi mouse e rendering grafico		Sistema QGraphicsScene / QGraphicsView per il disegno fluido
 Armadillo	Algebra lineare e calcolo numerico vettoriale		Vettori arma::vec2 e matrici per lo stato del motore fisico
 Boost.Math	Funzioni matematiche speciali e costanti ad alta precisione		Risolutore non lineare toms748_solve per la cicloide ideale
 Exprtk	Parsing sintattico ed esecuzione al runtime di formule matematiche		Valutazione di funzioni digitate nella GUI (es. $2x + x/2$)
 spdlog	Logging di sistema asincrono e formattato		Scrittura rapida dei log sia su console colorata sia su file di testo

LE SCELTE IMPLEMENTATIVE

Performance e Responsività

Il loop grafico e fisico deve girare stabilmente a 60 FPS (~16ms).

Integrità dei Dati Fisici

Validazione geometrica delle curve campionate.

Invarianza di Scala

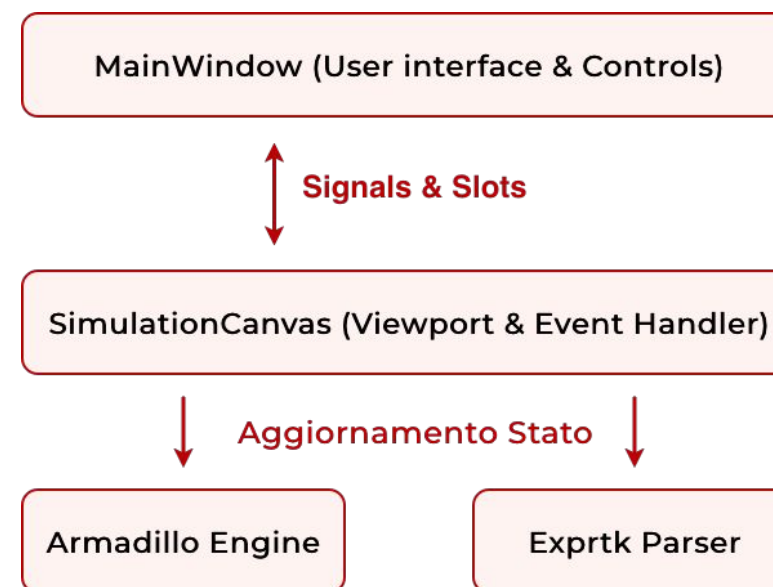
Correttezza della dinamica fisica indipendentemente dallo zoom pixel-metri impostato dall'utente.

STRUTTURA DEL SOFTWARE

MainWindow: Gestisce il pannello sinistro di controllo, i pulsanti, gli slider di scaling e la compilazione delle formule.

SimulationCanvas: Eredita da QGraphicsView e incapsula l'area logica di disegno, la palla fisica e il timer di refresh.

Comunicazione bidirezionale: I controlli utente aggiornano lo stato del Canvas, mentre gli eventi della simulazione pilotano l'aggiornamento dinamico dell'interfaccia tramite segnali.



Architettura software del progetto

GESTIONE DELL'INPUT UTENTE



Disegno Libero

Tracciamento manuale della traiettoria tramite mouse.

Filtro di post-processing: Pulizia dei punti ridondanti e garanzia di monotonia crescente lungo le ascisse ($x > 0$) per evitare curve non fisiche.



Input Analitico (Exprtk)

Inserimento testuale di formule matematiche.

Parser integrato: Compilazione a *runtime* tramite Exprtk e campionamento automatico per generare la curva corrispondente.

CALCOLO DEL TEMPO STIMATO

Modellazione del percorso: La curva campionata dall'utente è approssimata come una catena di piccoli segmenti rettilinei consecutivi, ciascuno di lunghezza d .

Velocità ai nodi: La velocità in entrata (v_1) e in uscita (v_2) da ogni segmento è calcolata applicando il principio di conservazione dell'energia meccanica:

$$v_2 = \sqrt{v_1^2 + 2g\Delta y}$$

(dove Δy è il dislivello verticale del tratto e la direzione della velocità è tangente al segmento).

Calcolo analitico del tempo (Δt): Considerando l'accelerazione costante su ogni tratto (moto uniformemente accelerato), il tempo di percorrenza parziale si ricava in forma chiusa dalla velocità media: $\Delta t = \frac{2d}{v_1 + v_2}$

MODELLO DI SIMULAZIONE

Integrazione Numerica dello Stato

La posizione della pallina lungo l'ascissa curvilinea S è ricavata risolvendo un sistema lineare discretizzato nel tempo:

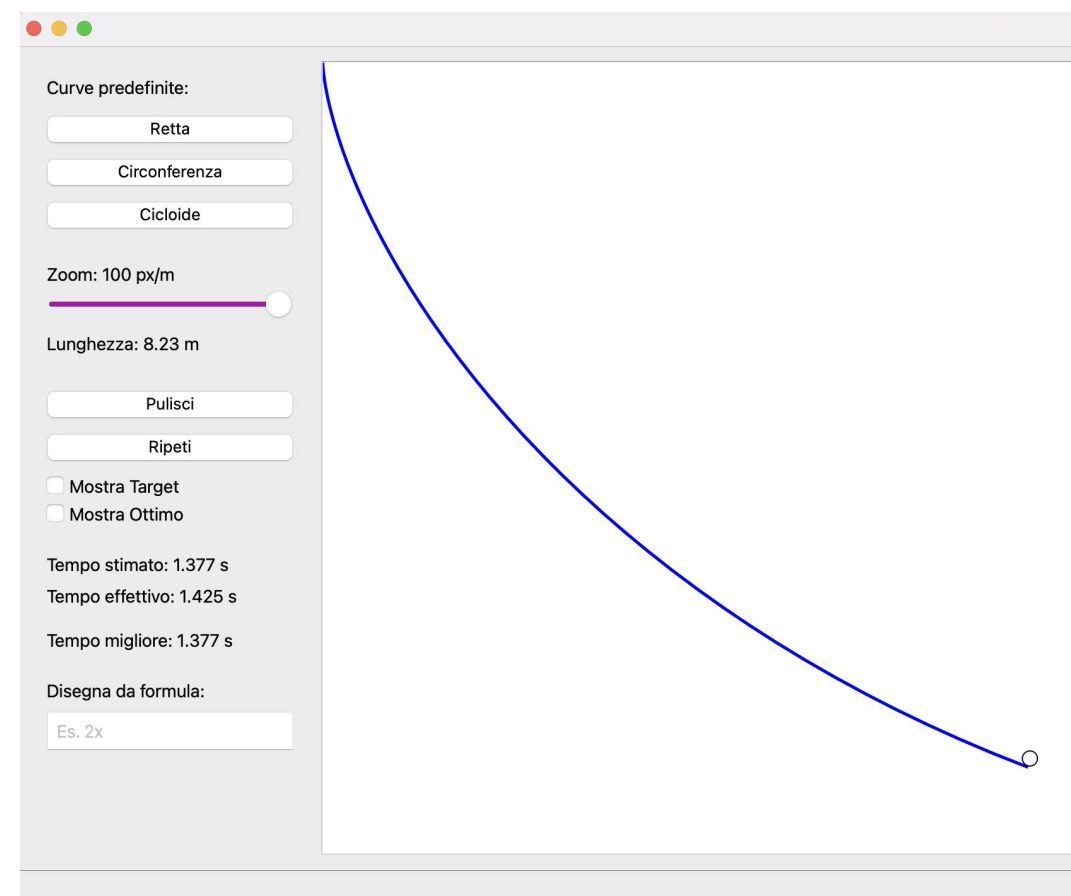
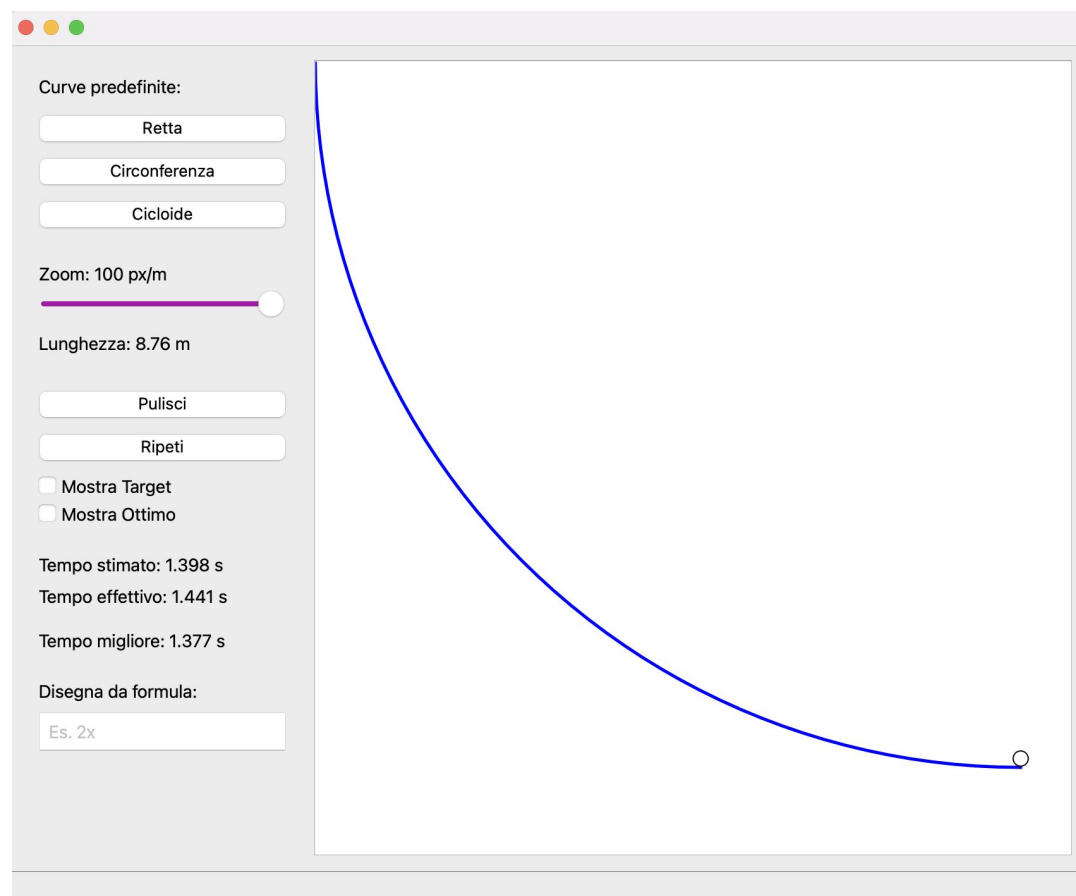
$$\begin{bmatrix} s(k+1) \\ v(k+1) \end{bmatrix} = \begin{bmatrix} 1.0 & \Delta t \\ 0.0 & 1.0 \end{bmatrix} \begin{bmatrix} s(k) \\ v(k) \end{bmatrix} + \begin{bmatrix} 0.0 \\ \Delta t \cdot \sin \theta \end{bmatrix} g$$

Calcolo dell'inclinazione: il seno dell'angolo è ricavato dinamicamente analizzando la pendenza del segmento locale su cui transita la sfera.

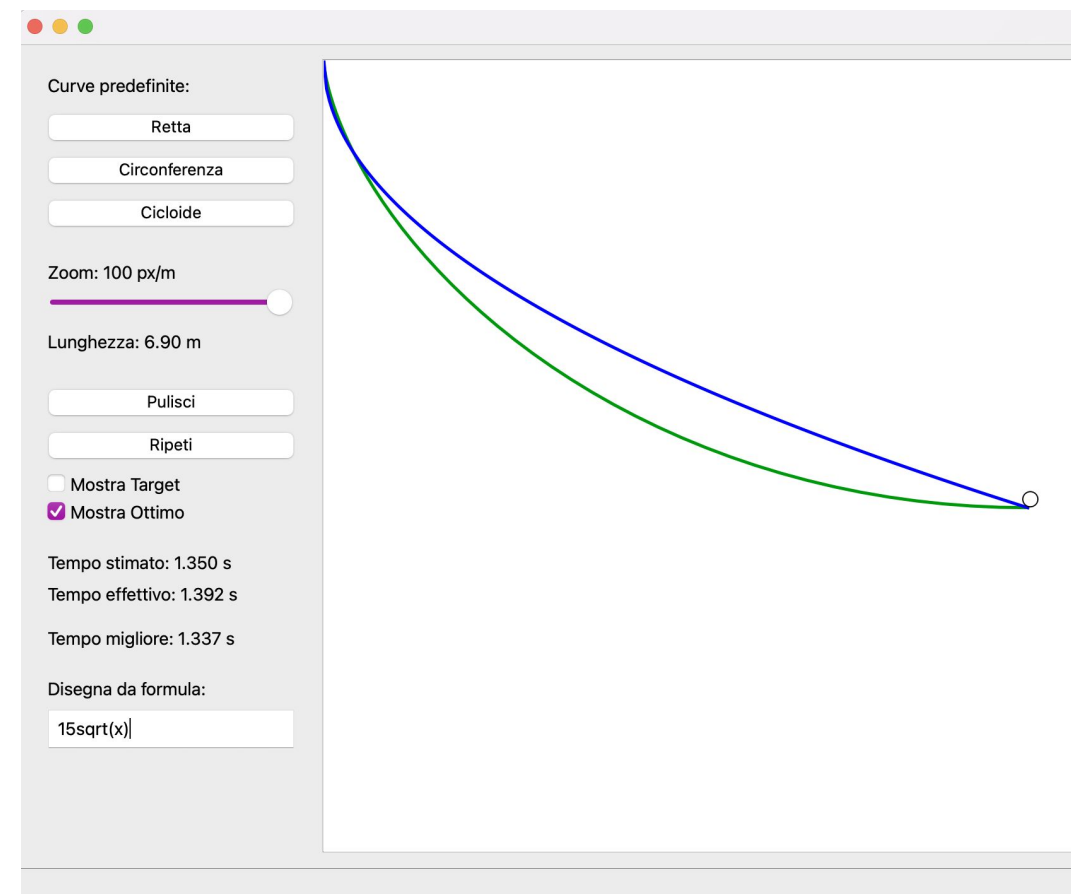
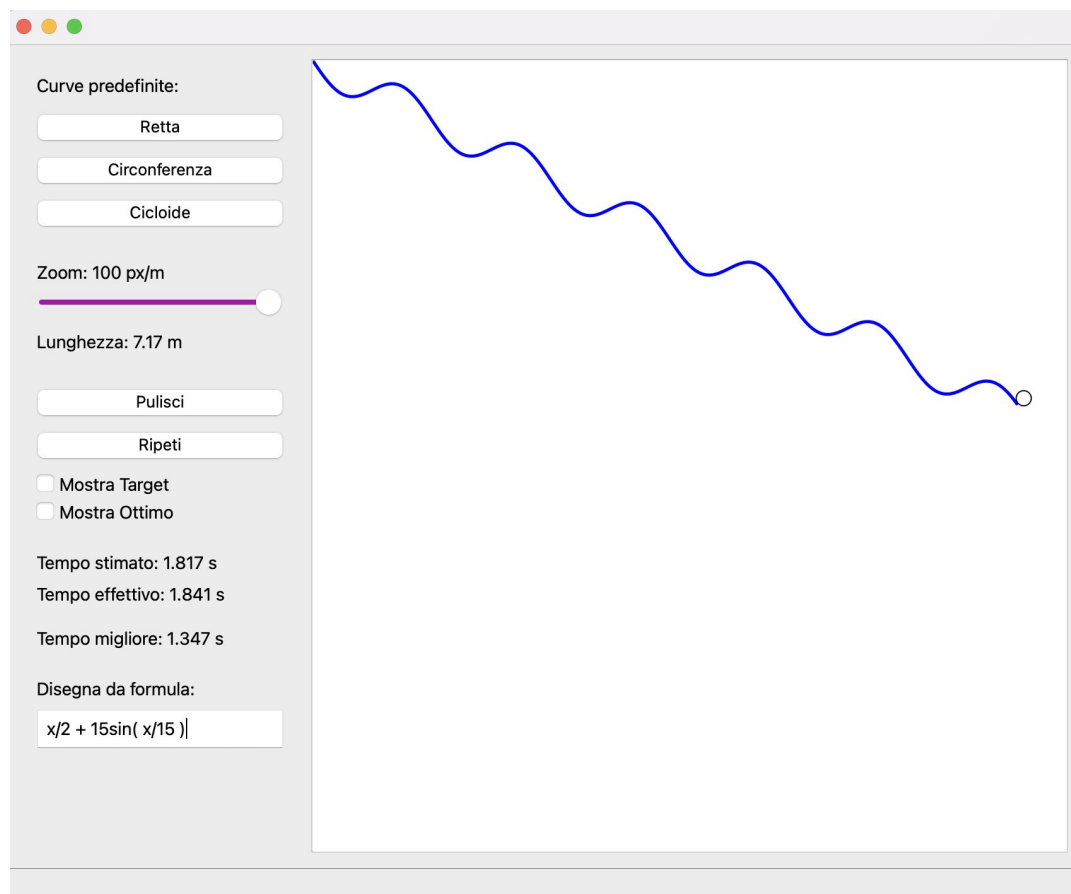
Intervallo temporale: Passo variabile basato sul tempo reale trascorso, limitato a un massimo di 50 ms in caso di lag.

Tolleranza all'errore numerico: L'errore di integrazione accumulato nel tempo è visivamente impercettibile e non inficia la fluidità o la coerenza visiva del moto.

CONCLUSIONI



CONCLUSIONI



CONCLUSIONI



Cattedra di Galileo Galilei all'Università
di Padova presso il Palazzo Bo

Unendo teoria matematica e ingegneria del software,
BDraw rende tangibili le sfide storiche della scienza
attraverso un'esperienza interattiva immediata.

<https://github.com/Mattia-Zanini/BDraw>